Credit Card Default Prediction using RBF-SVM and SHAP Explanation

IML Project

Problem Statement

Build an RBF-kernel SVM to predict credit card default using the default of credit card client dataset. Split the dataset into train and test sets and use the train set to build the SVM. Implement SHAP and use it to explain the predictions on the test set.

Index

Sr. No.	Title	Page No.
1	About the Dataset	2
2	Preprocessing and Visualization	4
3	About the Model	6
4	SHAP implementation	8
5	Model Explanations	9
6	Key Takeaways	10
7	References	10

1. About the Dataset

Link of the Dataset:
https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients

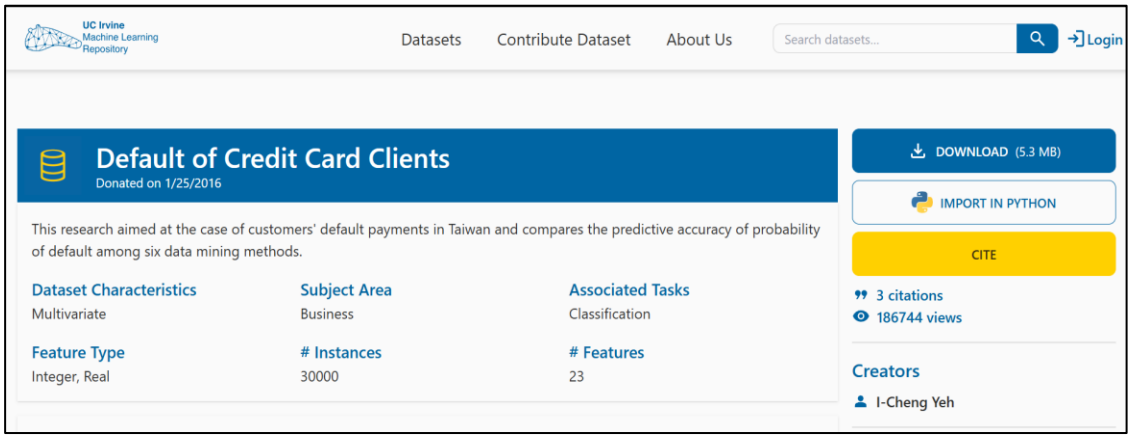

Fig1: Screenshot of the Dataset website

Dataset Information:

- Dataset size = 30,000 rows
- Number of Features = 23
- Target Variable = default payment next month (Yes=1, No=0)
- Used for classification purposes, in business-related domain

Features Info:

X1: Amount of the given credit (NT dollar): it includes both the individual consumer credit and his/her family (supplementary) credit

X2	Gender		X3	Education		X4	Marital Status
1	Male		**1**	graduate school		**1**	married
2	Female		**2**	university		**2**	single
			3	high school		**3**	others
			4	others			

Table 1: Showing values for features X2, X3, X4

X5: Age (in years)

Feature	Information
X6	repayment status in September 2005
X7	repayment status in August 2005
X8	repayment status in July 2005
X9	repayment status in June 2005
X10	repayment status in May 2005
X11	repayment status in April 2005

Table 2: Features X6-X11

Measurement Scale	Meaning
-1	pay duly
1	payment delay for 1 month
2	payment delay for 2 months
3	payment delay for 3 months
4	payment delay for 4 months
5	payment delay for 5 months
6	payment delay for 6 months
7	payment delay for 7 months
8	payment delay for 8 months
9	payment delay for 9 months

Table 3: Admissible values X6-X11 can take

Feature	Information	Feature	Information
X12	amount of bill statement in September 2005	**X18**	amount paid in September 2005
X13	amount of bill statement in August 2005	**X19**	amount paid in August 2005
X14	amount of bill statement in July 2005	**X20**	amount paid in July 2005
X15	amount of bill statement in June 2005	**X21**	amount paid in June 2005
X16	amount of bill statement in May 2005	**X22**	amount paid in May 2005
X17	amount of bill statement in April 2005	**X23**	amount paid in April 2005

Table 4: Features X12 to X23 and what they indicate

The dataset contains both numeric and categorical features. Numerical features include credit amount, age, bill amounts, and payment amounts, while categorical features include gender, education, marital status, and repayment status for six previous months.

2. Preprocessing and Visualization

There were **no missing values** in the dataset. Data cleaning involved removing **56 duplicate records** and correcting invalid categorical values. For example, education values (0, 5, 6) were remapped to 4 (Others), and marital status values (0) were remapped to 3 (Others). The dataset looks like this after pre-processing:

```
print(X.head())
print(y.head())
```

```
       X1  X2  X3  X4  X5  X6  X7  X8  X9  X10  ...    X14    X15    X16  \
0   20000   2   2   1  24   2   2  -1  -1   -2  ...    689      0      0
1  120000   2   2   2  26  -1   2   0   0    0  ...   2682   3272   3455
2   90000   2   2   2  34   0   0   0   0    0  ...  13559  14331  14948
3   50000   2   2   1  37   0   0   0   0    0  ...  49291  28314  28959
4   50000   1   2   1  57  -1   0  -1   0    0  ...  35835  20940  19146

     X17   X18    X19    X20   X21   X22   X23
0      0     0    689      0     0     0     0
1   3261     0   1000   1000  1000     0  2000
2  15549  1518   1500   1000  1000  1000  5000
3  29547  2000   2019   1200  1100  1069  1000
4  19131  2000  36681  10000  9000   689   679

[5 rows x 23 columns]
   Y
0  1
1  1
2  0
3  0
4  0
```

Fig 2: Preprocessed Dataset

The dataset is imbalanced, with the majority class being non-default clients. Stratified train-test split was used to maintain class proportion during training.

Fig 3: Class Distribution

We have also computed respective statistical metrics to show relationship between various features. We have used Pearson Coefficient for Numeric features and NMI index for Categorical features.

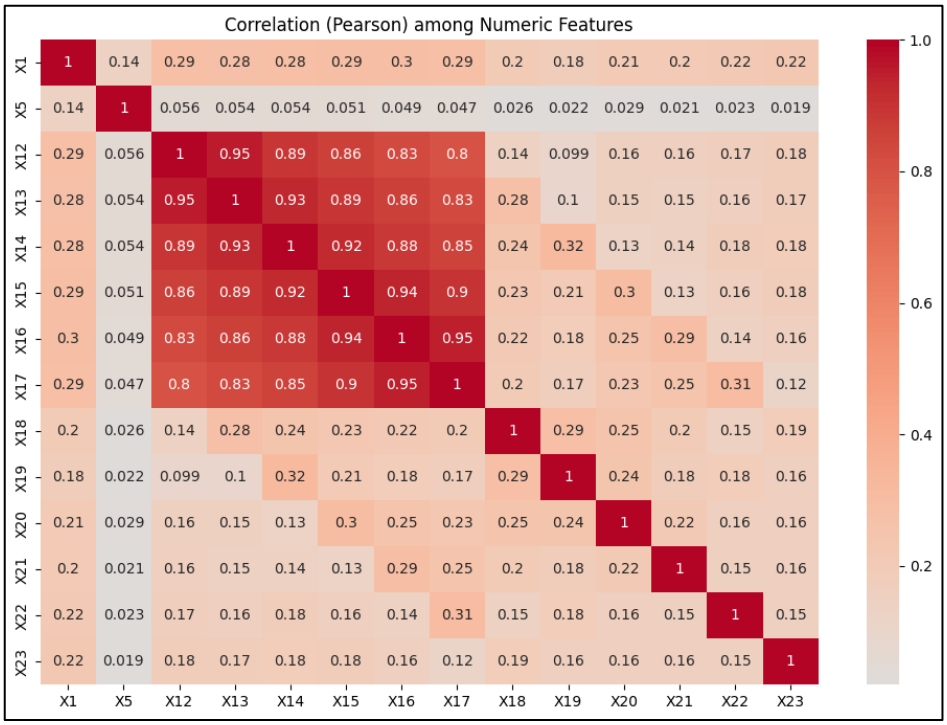

Fig 4: Pearson's Coefficient for Numeric Features

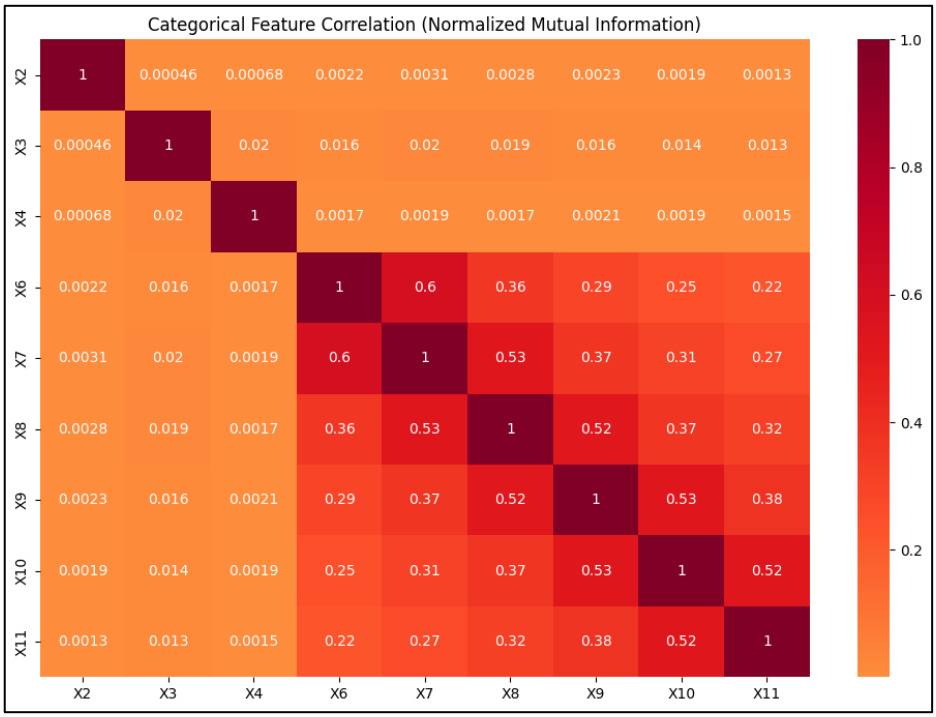

Fig 5: NMI for Categorical Feautres

3. About the Model

The classification model used is a Support Vector Machine (SVM) with an RBF (Radial Basis Function) kernel. This model is capable of capturing nonlinear relationships in the dataset. The implementation was carried out using the Scikit-learn library.

Model Hyperparameters:
- Kernel: RBF
- C: 1
- Gamma: 'scale'
- Random State: 42

Before training, categorical features (such as education, gender, and repayment status) were one-hot encoded, and numerical features were scaled using MinMaxScaler. The dataset was split into 70% training and 30% testing using stratified sampling.

RBF Kernel:
Since the dataset contains 23 features, and the relationships can be highly complex, it is best to use RBF Kernel as it projects the data into a higher dimensional plane which can be used to decide upon a suitable hyperplane later. It is given by:

$$K(x, x')=\exp(-\gamma\|x-x'\|^2)$$

Gamma is a hyperparameter and in our setup and we have used the standard value which is given by:

$$\gamma=1/(\text{number_of_features}*X.\text{var}())$$

Fine-Tuning:
Since the kernel is fixed, value of gamma is optimal, we tried to find the best C -> regularization parameter and chose 5 values: [0.01,0.1,1,10,100]
The results for C=0.01 and 100 are slightly lower than the others. (80% and 81% respectively)
The results are similar for C=0.1, 1, 10 with accuracy of 82% and hnce we chose C=1 for our model.

```
[[6699  298]
 [1314  673]]
              precision    recall  f1-score   support

           0       0.84      0.96      0.89      6997
           1       0.69      0.34      0.46      1987

    accuracy                           0.82      8984
   macro avg       0.76      0.65      0.67      8984
weighted avg       0.80      0.82      0.80      8984
```

Fig 6: Results for Best Hyperparameters

For other hyperparameters:

```
[[6848  149]
 [1606  381]]
              precision    recall  f1-score   support

           0       0.81      0.98      0.89      6997
           1       0.72      0.19      0.30      1987

    accuracy                           0.80      8984
   macro avg       0.76      0.59      0.59      8984
weighted avg       0.79      0.80      0.76      8984
```

```
[[6733  264]
 [1354  633]]
              precision    recall  f1-score   support

           0       0.83      0.96      0.89      6997
           1       0.71      0.32      0.44      1987

    accuracy                           0.82      8984
   macro avg       0.77      0.64      0.67      8984
weighted avg       0.80      0.82      0.79      8984
```

```
[[6664  333]
 [1295  692]]
              precision    recall  f1-score   support

           0       0.84      0.95      0.89      6997
           1       0.68      0.35      0.46      1987

    accuracy                           0.82      8984
   macro avg       0.76      0.65      0.68      8984
weighted avg       0.80      0.82      0.80      8984
```

```
[[6533  464]
 [1274  713]]
              precision    recall  f1-score   support

           0       0.84      0.93      0.88      6997
           1       0.61      0.36      0.45      1987

    accuracy                           0.81      8984
   macro avg       0.72      0.65      0.67      8984
weighted avg       0.79      0.81      0.79      8984
```

Fig 7: Confusion Matrix and Performance Metrics for other Hyperparameters

Each figure corresponding to C=0.01, 0.1, 10, 100 respectively.

4. SHAP Implementation

A custom implementation of the **SHAP (SHapley Additive exPlanations)** method was developed from scratch. The implementation focuses on explaining the SVM model's predictions for selected top features using exact Shapley value computation for a manageable subset of features.

Pseudocode:

1. Select top-k important features using Random Forest feature importance.
2. Compute background vector as the mean of each feature from the training data.
3. For each subset of the selected features:
   a. Replace features not in the subset with background values.
   b. Evaluate the model's prediction using the SVM decision function.
4. Compute marginal contributions for each feature and subset.
5. Apply Shapley weight formula to aggregate feature contributions into final SHAP values.
6. Sum of base value and SHAP values approximates the model prediction.

Mathematical Analogy:

Assuming the following:

Model Function $f(x)$

instance of interest $\bar{x}$

N features

background x^*

Total number of combinations/coalitions = 2^N (Power Set of N).

The empty set represents the base value.

$$\therefore BaseValue = f(x^*)$$

Now for every subset say S we replace the corresponding features of x^* as that of $\bar{x}$ and pass it to the the black-box model $f()$ and calculate the marginal contributions:

$$\Delta_i S = f(S \cup i) - f(S)$$

Across all the subsets in an orderly manner, getting $2^{(N-1)}$ marginal contributions

Because if we drop i^{th} feature, then there are $N-1$ combinations.

Next, we estimate the Shapley weights for each of these contributions as:

$$w(S) = \frac{|S|!\,(N-|S|-1)!}{N!}$$

Therefore, SHAP is weighted sum given by:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(N-|S|-1)!}{N!} \left[f(S \cup \{i\}) - f(S) \right]$$

Fig 8: Written Explanation for SHAP formula

About the implementation, since after one-hot encoding the number of features were 84, we have restricted ourselves to only 15 features as crashes were frequent and have used 2D slicing pattern to avoid Colab limitations. For example, we had initially used a 1D list to store all possible combinations, but if the size grows, it crashes, hence, we have broken down the 1D list into fixed size chunks and appended the remaining as another 1D element, creating a 2D array.

We renamed the features of the form "a(number)" so when we perform coalitions, we can easily identify the individual features from the combination. Example -> a2a3a1 is a2, a3, a1 which helps us to easily replace corresponding feature values and cross-check. We store the marginal contributions, as a 2D matrix and later, we have increased it as a 3D array as described above. We perform basic col-wise and row-wise operations to get our desired marginal contributions and later we compute the SHAP values using the weights which were pre-computed.

5. Model Explanations

To understand model behavior, SHAP values were computed for selected instances from the test data. Each feature's SHAP value indicates how much that feature contributed to increasing or decreasing the probability of credit card default.

Feature	Instance 1	Instance 2	Instance 3
X5	**0.00068**	0.00109	-0.00049
X1	**-0.00065**	**-0.00086**	-0.00011
X12	0.00036	-0.00000	-0.00036
X6_2	-0.00150	**0.02394**	**-0.00227**
X13	-0.00007	-0.00000	0.00008
X14	0.00010	-0.00005	-0.00014
X15	0.00008	-0.00003	-0.00010
X17	0.00012	-0.00004	-0.00015
X16	-0.00006	0.00005	0.00008
X18	0.00004	0.00006	0.00004
X19	0.00000	0.00001	-0.00000
X20	0.00002	0.00005	0.00004
X23	0.00007	0.00013	**0.00016**
X21	0.00006	0.00008	0.00011
X22	0.00000	-0.00001	0.00001

Table 5: SHAP values of Instances

Feature	1	2	3
X5	56	22	32
X1	100000	50000	210000
X12	97549	50974	355
X6	0	2	-2
X13	102864	51688	975
X14	98969	50024	410
X15	70352	29381	0
X17	68538	28435	0
X16	68186	28957	0
X18	9300	3000	979
X19	0	0	412
X20	2500	800	0
X23	2600	775	0
X21	2700	808	0
X22	2700	1200	0

Table 6: Actual Values of Instances

Instance 1:

For the first instance, we can say that feature X5 which is age contributed most positively for the prediction class 0 meaning that age for this instance which is "56" is favourable and aligns that the person will pay the credit card payment next month. Generally, more the age, more the responsibility and hence is explains the prediction. Similarly, we can see which other features have how much effect on the prediction, as X1 which contributed by pulling down the prediction and it represents the amount of credit given which for this case was 100000 dollars.

Instance 2:

Similarly, we can argue that why this instance could not pay the credit card amount of next month and was given the class 1. This was because most features contributed positively with X6_2 the most which is payment is due for the month of September 2005 from the past 2 months. This explains why the model said the person won't be able to pay next month when his earlier payments have not been done.

Instance 3:

Similarly, we can also explain why in this case, the model predicted the person will be able to pay the amount. Here we see X6_2 again but its contribution is opposite meaning that the person had paid their amount earlier for September 2005. However, we also see X23 which is previous amount in April which slightly shifts the prediction to class 1 but overall, the SHAP values contribute negatively.

Observations:

We observe that if the label is class 0, then the SHAP should contribute negatively to the base value which is consistent with the values. Similarly, if the label is class 1, then SHAP values should overall contribute positively thereby lifting the base value to the desired prediction which is also consistent.

6. Key Takeaways

- The UCI credit card default dataset is clean and structured, requiring minimal preprocessing.
- The RBF-SVM model is suitable for capturing complex nonlinear relationships in data.
- The custom SHAP implementation provides interpretable and exact explanations for top features.
- Features like recent payment behavior, bill amounts, and credit limits have significant influence on default prediction.
- The explainability framework developed can be reused for other models and datasets.
- Future work includes optimizing hyperparameters, handling class imbalance, and visualizing SHAP results for multiple samples.

7. References

[1] UCI Machine Learning Repository: Default of Credit Card Clients Dataset